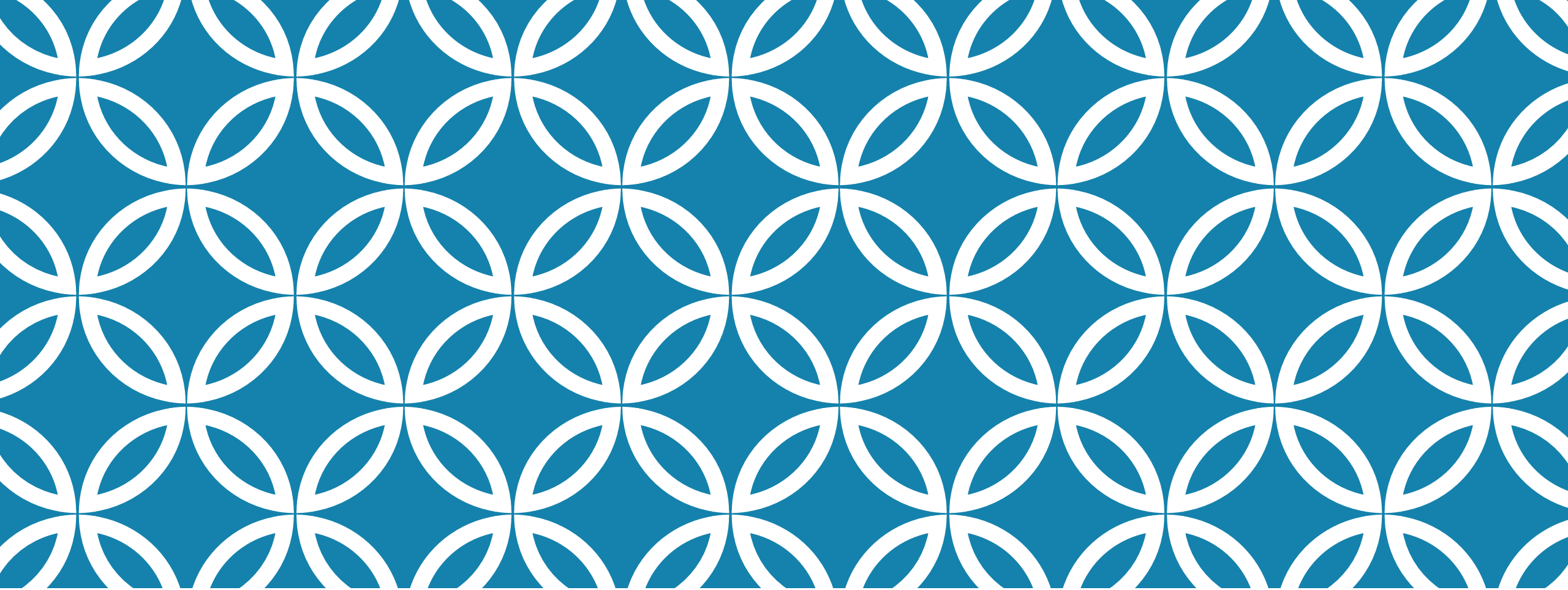


OPERATOR PRECEDENCE PARSER

OPERATOR PRECEDENCE PARSING

Parsing of an **operator grammar**

It is a **bottom-up parser**

Operator grammar is a grammar where productions cannot contain two or more consecutive NTs in any RHS alternative

Non terminals occurring in RHS string are separated by one or more terminal symbols

All terminal symbols occurring in RHS strings are called operators of the grammar

Not an operator grammar

$$E \rightarrow E\ op\ E \mid id$$

$$op \rightarrow + \mid *$$

An operator grammar

$$E \rightarrow E + E \mid E * E \mid id$$

Is the grammar below an operator grammar?

S → A a B | A

A → B b | B

B → S

Operator precedence operators

$o1 <\cdot o2$ → *o1 is having less precedence than o2 i.e. o2 has higher precedence*

$o1 \cdot> o2$ → *o1 is having higher precedence than o2*

$o1 =. o2$ → *o1 is having same precedence as o2*

OPERATOR PRECEDENCE TABLE

Operator precedence parser can parse some of the ambiguous grammar by constructing an operator precedence table

E → E + E | E * E | id

	id	**+**	*	**$**
id		·>	·>	·>
+	<·	·>	<·	·>
*	<·	·>	·>	·>
$	<·	<·	<·	

EXAMPLE

We parse the string

a) id + id * id

b) id + id + id

Using the grammar before and following operator precedence table

E → E + E | E * E | id

> id + id * id

Stack	Input	Action
$	id + id * id $	$ <· id, shift
$id	+ id * id $	id ·> +, reduce E → id
$	+ id * id $	$ <· id, shift
$+	id * id $	+ <· id, shift
$+id	* id $	id ·> +, reduce E → id
$+	* id $	+ <· *, shift
$+*	id $	* <· id, shift
$+*id	$	id ·> $, reduce E → id
$+*	$	* ·> $, reduce E → E*E
$+	$	+ ·> $, reduce E → E+E
$	$	Accept

	id	+	*	$
id		·>	·>	·>
+	<·	·>	<·	·>
*	<·	·>	·>	·>
$	<·	<·	<·	

Note:
if op1 <· op2, shift operation
Otherwise reduce operation

JUST TO NOTE…

(=·)
$ <· (
id ·>)
) ·> $
(<· (
$ <· id
id ·> $
) ·>)
(<· id

The rules for operator precedence will also be based on the precedence of operators

EXERCISE

Create an operator precedence table for the grammar

$$E \rightarrow \quad E+E \mid E-E \mid E*E \mid E/E \mid E\hat{}E \mid id$$

Using the operator precedence table, reduce the string

id * id ^ id + id

Hence, we make

$\theta <\cdot$ unary-minus → *for any operator*

unary-minus $\cdot>$ θ → *if unary-minus has higher precedence than* θ

unary-minus $<\cdot$ θ → *if unary-minus has lower (or equal) precedence than* θ

ADVANTAGES

- Simple
- Powerful enough for expressions in programming languages

DISADVANTAGES

Cannot handle unary minus